

```

1 pool:
2   name: Hosted Ubuntu 1604
3
4 steps:
5
6 - script: |
7   docker build -t osfy-proj:latest .
8   docker tag osfy-proj:latest vivekiiitbcontainerregistry.azurecr.io/$(Build.Repository.Name):latest
9   docker image ls -a
10  docker login -u vivekiiitbcontainerregistry -p J+KmvV6xCpW/ByrVzucNrUYWvN8Qgvp1
    vivekiiitbcontainerregistry.azurecr.io
11  docker push vivekiiitbcontainerregistry.azurecr.io/$(Build.Repository.Name):latest
12  displayName: 'Docker build and push'

```

Figure 6: Definition of *azure-pipelines.yml* file

The contents of *Dockerfile* are given below:

Running the website locally

The current directory is as shown below:

```
azureuser@Kube:~/OSFY-project/projects ls
Dockerfile index.html
```

In the *index.html* file, there is a basic markup titled DevOps and a heading. The contents of *index.html* are given below:

```

<!doctype html>
<html>
  <head>
    <title>Devops</title>
  </head>
  <body>
    <h1>Hello, AzureDevops!</h1>
  </body>
</html>

```

First *pull* the Nginx Alpine-based docker image, and then copy *index.html* to the image in the following *Dockerfile*.

```

FROM nginx:alpine
LABEL author="Vivek Gupta"
COPY index.html /usr/share/nginx/html
EXPOSE 80 443
ENTRYPOINT [ "nginx", "-g", "daemon
off;" ]

```

For running the application locally, we need to follow the steps given below.

1. Build the Docker image manually:

```
$ docker build -t osfy-proj:latest
```

2. Run the Docker container:

```
$ docker run -d -p 80:80 osfy-proj:latest
```

Now you can see the website getting served on port 8080, as shown in Figure 1.

Building the container using the Azure DevOps pipeline

To run this website in the Azure pipeline, first, the Azure Container Registry resource needs to be created.

1. Visit the website <http://shell.azure.com/> and login with the Azure subscription, as shown in Figure 2.
2. Now create a new resource group, named *theResourceGroup* by giving the following command:

```
$ az group create --name theResourceGroup --location eastus
```

3. Then create the Azure Container Registry resource in the resource group *theResourceGroup*. Make sure to name it uniquely:

```
$ az acr create --resource-group theResourceGroup --name vivekiiitbcontainerregistry --sku basic
```

4. The *adminUserEnabled* property on *vivekiiitbcontainerregistry* is disabled by default. To enable it, you must run the following command:

```
$ az acr update --name vivekiiitbcontainerregistry --admin-enabled true
```

The screenshot displays the Azure DevOps web interface. On the left is a navigation sidebar with options like Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, and Test Plans. The main area shows the details of a pipeline run. At the top, it says '#20200928.1 Azure DevOps Pipeline added on first'. Below this is a 'Summary' section indicating it was triggered by Vivek Gupta. A table-like summary shows repository and version information, time started, and related items. At the bottom, a 'Jobs' section shows a single job named 'Job' with a status of 'Queued'.

Figure 7: Screen when we run the pipeline